El proyecto explicado fácil

Reconocedor de letras con Cadenas Ocultas de Markov · Procesos Estocásticos (603504) · Unillanos

1. ¿Qué hace el proyecto? (en una frase)

> **Un programa donde dibujas una letra con el mouse y, mientras la vas dibujando, el programa adivina qué letra es, muestra qué tan seguro está, y dibuja con línea punteada cómo cree que la vas a terminar.**

Es como el teclado del celular cuando te sugiere la palabra antes de que termines de escribirla, pero con dibujos de letras.

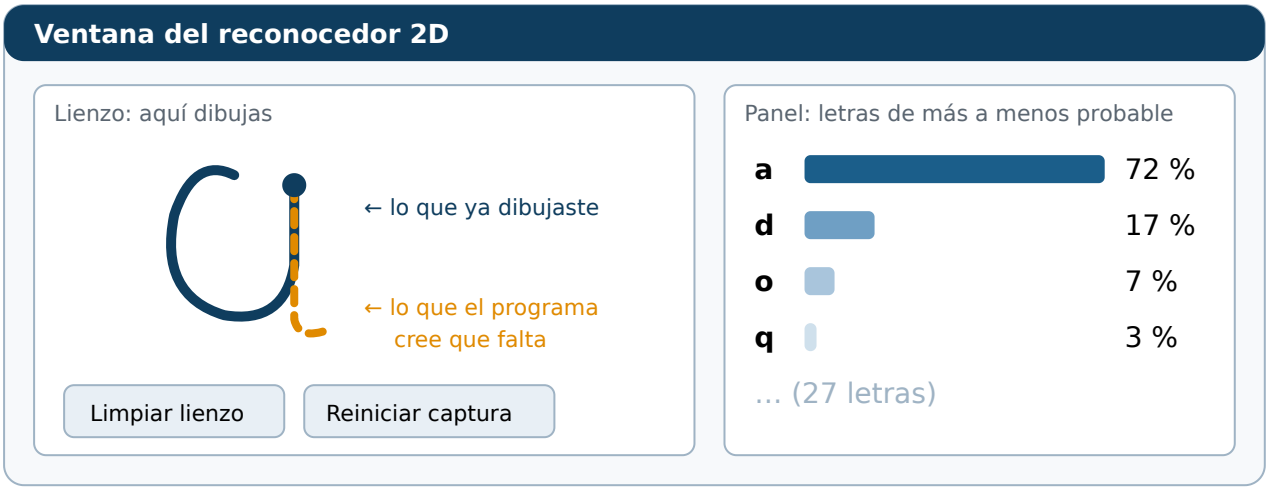
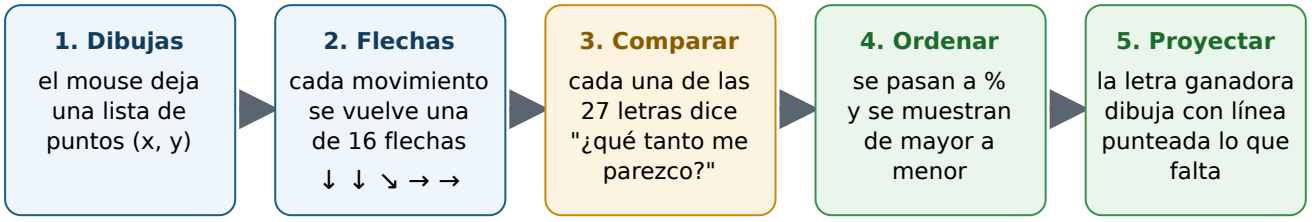


Figura 1. Así se verá la ventana 2D: lienzo a la izquierda, lista de letras a la derecha.

Lo que pide el profesor, resumido

Parte	Qué es
Ventana A: Menú	Pantalla de inicio para elegir modo 2D o 3D.
Ventana B: 2D	Dibujar en una hoja plana (como la figura 1).
Ventana C: 3D	Lo mismo, pero dibujando sobre una **esfera** que se puede girar y acercar.
Letras	Las 27 minúsculas del español: a, b, c, …, n, **ñ**, o, …, z.
Regla de oro	Las matemáticas (Cadenas Ocultas de Markov) se programan **desde cero**. Prohibido usar librerías que ya lo hagan; solo se permite NumPy para hacer cuentas.
Entrega	Código en GitHub + ZIP al correo del profe + **sustentación**: demo en vivo y explicar las fórmulas a mano en el tablero.

2. ¿Cómo funciona por dentro? (5 pasos)



Esto se repite cada vez que mueves el mouse, por eso es "en tiempo real".

Figura 2. El recorrido de un dibujo dentro del programa.

¿Y de dónde sabe el programa cómo es cada letra?

Antes de usar la app, hay un paso aparte llamado **entrenamiento**. Se le muestran al programa muchos ejemplos de cada letra (por ejemplo, 100 "a" dibujadas por distintas personas, sacadas del dataset compartido *Glifo*), y él aprende "cómo suele ser" cada una. Lo aprendido se guarda en archivos, y la app solo los carga al abrir. **La app no aprende mientras la usas**: solo reconoce.

La idea clave: fases ocultas y pistas visibles

Toda letra se dibuja por **fases**. Por ejemplo, una "L": primero *bajar*, después *ir a la derecha*. El programa **no ve** en qué fase estás; solo ve **hacia dónde se mueve el mouse** (las flechas). Con esas pistas adivina la fase y la letra.

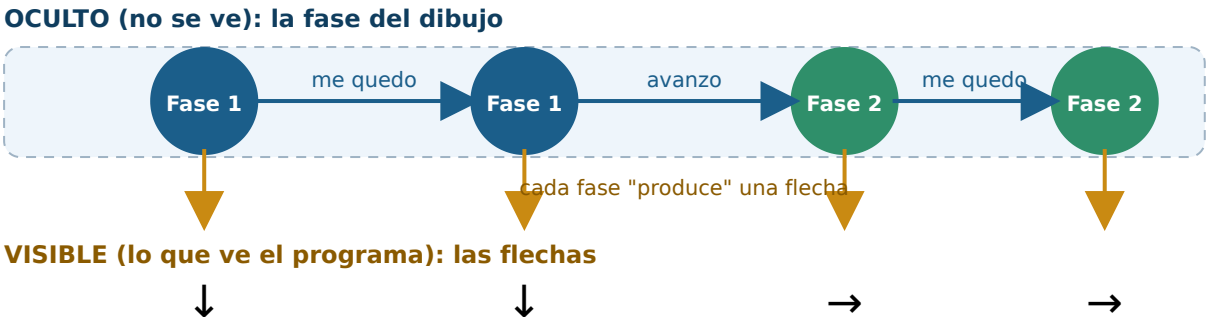


Figura 3. Una "L" vista como Cadena Oculta de Markov.

Eso es justamente una **Cadena Oculta de Markov**: algo escondido que avanza por pasos (la fase) y que deja pistas visibles (las flechas). Es como adivinar el clima de otra ciudad solo viendo si tu amigo de allá lleva paraguas.

3. Temas del curso que usa el proyecto

BASE probabilidad que ya conoces MARKOV cadenas de Markov HMM cadenas ocultas y sus algoritmos

Proceso estocástico BASE

Algo que cambia con el tiempo de forma **aleatoria** (no 100 % predecible). Se describe como una secuencia de variables aleatorias X_1, X_2, X_3, …

En el proyecto: el dibujo de una letra. Nadie dibuja la "a" exactamente igual dos veces; cada trazo es una secuencia de movimientos con algo de azar.

Probabilidad condicional BASE

La probabilidad de que pase A **sabiendo** que ya pasó B. Se escribe `P(A | B)`.

En el proyecto: casi todo es condicional: "probabilidad de ver estas flechas *si* la letra es una a" → `P(flechas | a)`.

Teorema de Bayes BASE

Permite "voltear" una probabilidad condicional: pasar de `P(B | A)` a `P(A | B)`. Fórmula: `P(A|B) = P(B|A)·P(A) / P(B)`.

En el proyecto: los algoritmos calculan `P(flechas | letra)`, pero el panel necesita lo contrario: `P(letra | flechas)`, es decir, el porcentaje de cada letra. Bayes hace esa conversión, y así los 27 porcentajes suman 100 %.

Cadena de Markov y propiedad de Markov MARKOV

Un proceso que salta entre **estados**, donde el siguiente estado **solo depende del estado actual**, no de todo el pasado ("no tiene memoria").

En el proyecto: los estados son las **fases** del dibujo de una letra. Para saber a qué fase pasarás, solo importa en qué fase estás ahora.

Ejemplo cotidiano: el clima. Si hoy llueve, mañana probablemente también; no importa cómo estuvo hace una semana.

Matriz de transición A MARKOV

Tabla con la probabilidad de pasar de cada estado a cada otro: $a_{ij} = P(\text{ir al estado } j \mid \text{estoy en } i)$. Cada fila suma 1 (matriz estocástica).

En el proyecto: dice si en cada fase "me quedo" o "avanzo a la siguiente fase". Cada letra tiene su propia matriz A.

Distribución inicial π MARKOV

Probabilidad de en qué estado empieza la cadena.

En el proyecto: π = (1, 0, 0, ...): todo dibujo empieza en la fase 1.

Estados absorbentes y cadena izquierda-derecha MARKOV

Un estado **absorbente** es uno del que nunca se sale. En una cadena **izquierda-derecha** solo se puede quedar o avanzar, nunca retroceder.

En el proyecto: las fases de una letra solo van hacia adelante (no vuelves de la fase 3 a la 1). La última fase es absorbente: ahí termina la letra.

Tiempo de permanencia (distribución geométrica) MARKOV

Cuántos pasos seguidos se queda la cadena en un mismo estado. En promedio: $1 / (1 - a_{ii})$.

En el proyecto: sirve para la **línea punteada**: si en la fase 3 la probabilidad de quedarse es 0.75, esa fase dura en promedio 1/(1−0.75) = 4 pasos, y así se sabe qué tan largo dibujar cada parte que falta.

Cadena Oculta de Markov (HMM) HMM

Una cadena de Markov cuyos estados **no se ven**; solo se ven **observaciones** (pistas) que cada estado produce. Se describe con tres tablas: $\lambda = (A, B, \pi)$.

En el proyecto: es el tema central. Cada una de las 27 letras es un HMM: estados ocultos = fases; observaciones = flechas.

Matriz de emisión B HMM

Probabilidad de que cada estado produzca cada observación: $b_j(k) = P(\text{ver la pista } k \mid \text{estoy en el estado } j)$.

En el proyecto: dice qué flechas son típicas en cada fase. En la fase "bajar" de la L, la flecha ↓ tiene probabilidad alta.

Verosimilitud HMM

Qué tan bien un modelo "explica" unos datos: `P(datos | modelo)`. Entre más alta, mejor encaja.

En el proyecto: se calcula para las 27 letras con el mismo dibujo; la de mayor verosimilitud es la letra que gana.

Algoritmo Forward (hacia adelante) HMM

Calcula la verosimilitud `P(O | λ)` de forma rápida: en vez de revisar todos los caminos posibles uno por uno (millones), va acumulando paso a paso cuánta probabilidad llega a cada estado ($\alpha_t(i)$).

En el proyecto: es el corazón de la clasificación en tiempo real. Con cada flecha nueva se actualiza un paso. **Se explica en el tablero.**

Algoritmo Backward (hacia atrás) HMM

Igual que Forward pero desde el final hacia el principio ($\beta_t(i)$).

En el proyecto: no se usa en la app; solo en el entrenamiento, junto con Forward.

Algoritmo de Viterbi HMM

Encuentra la **secuencia de estados ocultos más probable**. Es como Forward, pero cambia la **suma** por el **máximo** y al final "devuelve el camino" (retroceso).

En el proyecto: dice en qué fase de la letra vas, para saber desde dónde dibujar la línea punteada. **Se explica en el tablero.**

Baum-Welch (máxima verosimilitud / EM) HMM

Método para **aprender** A, B y π a partir de ejemplos. Empieza con valores aproximados y los mejora repetidamente "contando" (en promedio) cuántas veces se pasó por cada fase y qué flechas salieron ahí. Cada vuelta mejora o iguala la verosimilitud.

En el proyecto: es el **entrenamiento** offline: con ~100 dibujos de cada letra se obtiene su modelo λ y se guarda en un archivo.

Predicción / proyección HMM

Usar el modelo para estimar lo que **viene después**: dada la fase actual, propagar con la matriz A los pasos futuros y tomar las observaciones más probables.

En el proyecto: la línea punteada (autocompletado). Si la letra que va ganando cambia, la línea cambia también.

Temas de apoyo (no son de Markov, pero se usan)

Tema	Descripción corta	Para qué
Cuantización (código de cadena)	Convertir algo continuo en pocas categorías: cada movimiento del mouse se redondea a una de 16 direcciones.	Los HMM del curso trabajan con un número finito de símbolos.
Coordenadas esféricas (θ, φ)	Describir un punto de una esfera con dos ángulos, como latitud y longitud.	Medir las direcciones al dibujar sobre la esfera 3D.
Escalamiento / logaritmos	Multiplicar muchas probabilidades pequeñas da números tan chicos que el computador los vuelve 0. Se normaliza en cada paso o se usan logaritmos.	Que los cálculos no se "rompan" con dibujos largos.
Programación orientada a objetos	Organizar el código en clases separadas (matemáticas, dibujo, ventanas).	El profesor lo evalúa (arquitectura limpia).

4. Todo junto en un ejemplo pequeño

Supongamos que solo existen dos letras, **"L"** (bajar y luego derecha) y **"Γ"** (derecha y luego bajar), y el usuario dibujó ↓ ↓ →.

Paso	Qué pasa	Tema del curso
1	El dibujo se convierte en flechas: ↓ ↓ →	Cuantización
2	Cada letra tiene su modelo λ = (A, B, π) aprendido antes con ejemplos	HMM + Baum-Welch
3	Forward calcula: P(↓↓→ | L) = 0.122 y P(↓↓→ | Γ) = 0.027	Forward / verosimilitud
4	Bayes convierte a porcentajes: **L = 82 %**, Γ = 18 %	Bayes
5	Viterbi dice que el camino más probable fue fase 1 → fase 1 → fase 2: ya vamos en "ir a la derecha"	Viterbi
6	Como la fase 2 de la L sigue a la derecha, la línea punteada continúa hacia la derecha	Predicción + tiempo de permanencia

La app real hace exactamente esto, pero con **27 letras**, unas **8 fases** por letra, **16 direcciones**, y lo repite cada vez que mueves el mouse.

5. Resumen para recordar

> **Markov:** el siguiente paso solo depende del actual.
> **Oculta:** no vemos la fase, solo las flechas que deja.
> **Forward:** ¿qué tanto se parece mi dibujo a cada letra? → ranking.
> **Bayes:** convierte eso en porcentajes.
> **Viterbi:** ¿en qué fase voy? → desde dónde proyectar.
> **Baum-Welch:** aprender cada letra con ejemplos (antes de usar la app).
> **Proyección:** dibujar lo que falta de la letra ganadora.